Understanding CTS of SPM design using OpenLane

INTRODUCTION:

The Clock Tree Synthesis (CTS) Lab in OpenLane focuses on building a balanced and optimized clock distribution network for a digital ASIC design. CTS is performed after placement and before routing, with the goal of delivering the clock signal to all sequential elements with minimal skew, latency, and power consumption. OpenLane automates CTS using open-source tools to insert clock buffers and inverters while meeting timing and design rule constraints. A well-designed clock tree is essential for reliable synchronous operation and timing closure.

APPLICATION:

The CTS Lab in OpenLane is used to generate an efficient clock distribution network for digital designs such as processors, controllers, and SoCs. It helps students and designers understand how clock skew, latency, and buffer insertion affect timing performance and power. CTS is widely applicable in educational ASIC flows, RISC-V cores, and open-source silicon implementations using Sky130. Proper clock tree synthesis ensures robust timing performance in subsequent routing and signoff stages

OBJECTIVE:

After completing this lab, you will be able to:

• Understand the role of CTS in the RTL-to-GDSII flow

• Generate a balanced clock tree using OpenLane

• Analyse clock skew, latency, and buffer insertion

• Verify clock network connectivity and timing

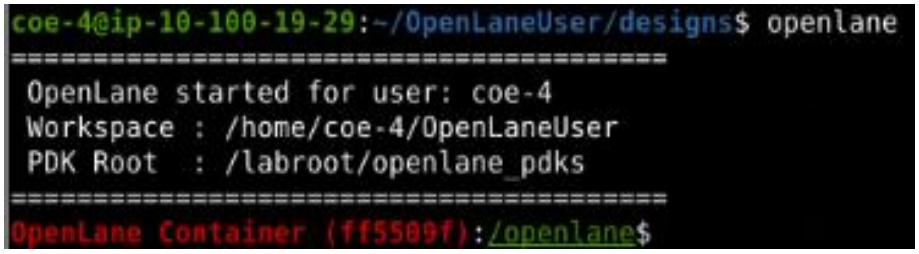
• Prepare the design for routing and timing closure

PROCEDURE TO CREATE A LAB:

Step 1: Set up OpenLane flow

1.1: Invoke OpenLane in the terminal using the command: `openlane`

efabhub.in



1.2: Open folder- designs: `cd designs`

Step 2: Set up Project Directory

2.1: Create a directory for your lab: `mkdir <project_name>`

2.2: Open lab directory: `cd <project_name>`

2.3: Create directory for project: `mkdir spm`

2.4: Open project directory: `cd spm`

2.5: Inside project directory, create folder: `mkdir src`

2.6: Open folder: `cd src`

Step 3: Create Design file

3.1: Open vi editor for design with command: `vi spm.v`

Note: for RTL code refer the previous document of Demonstration of floorplan using openlane

Step 4: Create .json file

4.1: Open vi editor: `vi config.json`

```
{
    "DESIGN_NAME": "spm",
    "VERILOG_FILES": "dir::src/*.v",
    "CLOCK_PERIOD": 10,
    "CLOCK_PORT": "clk",
    "CLOCK_NET": "ref::$CLOCK_PORT",
    "FP_PDN_VOFFSET": 7,
    "FP_PDN_HOFFSET": 7,
    "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
    "FP_PDN_SKIPTRIM": true,
    "pdk::sky130*": {
        "FP_CORE_UTIL": 45,
```

eChipHub.in

```
        "scl::sky130_fd_sc_hd": {
            "CLOCK_PERIOD": 10
        },
        "scl::sky130_fd_sc_hdll": {
            "CLOCK_PERIOD": 10
        },
        "scl::sky130_fd_sc_hs": {
            "CLOCK_PERIOD": 8
        },
        "scl::sky130_fd_sc_ls": {
            "CLOCK_PERIOD": 10,
            "MAX_FANOUT_CONSTRAINT": 5
        },
        "scl::sky130_fd_sc_ms": {
            "CLOCK_PERIOD": 10
              }
    },
    "pdk::gf180mcu*": {
        "CLOCK_PERIOD": 24.0,
        "FP_CORE_UTIL": 40,
        "MAX_FANOUT_CONSTRAINT": 4,
        "PL_TARGET_DENSITY": 0.5
    }
}
```

Step 5: Use the following commands to run the flow:

OpenLane to start the design flow in interactive mode

i) `flow.tcl -interactive`

Used in the OpenLane interactive shell to prepare your design for the digital IC design flow.

ii) `prep -design spm`

Runs yosys synthesis on the current design.

edinpub.in

iii) run_synthesis

To initiate floorplanning

iv) run_floorplan

To initiate placement

V) run_placement

To initiate cts

V) run_cts

```
% run_placement
[STEP 7]
[INFO]: Running Global Placement (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/placement/7-global.log)...
[STEP 8]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/placement/8-gpl_sta.log)...
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/placement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/placement/10-detailed.log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/placement/11-dpl_sta.log)...
% run_cts
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/cts/12-cts.log)...
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.01.29_06.44.46/logs/cts/13-cts_sta.log)...
%
```

Step 6: View Floorplan using klayout:

Locate the DEF file

i)designs/spm/runs/RUN_2026.01.23_05.16.33/results/cts/spm.def

ii)designs/spm/runs/RUN_2026.01.23_05.16.33/tmp/merged.nom.lef

Open in klayout in the same directory where spm.def is located

iii) klayout spm_cts.def merged.nom.lef

NOTE: keep both file cts.def and merged.nom.lef in the same directory.

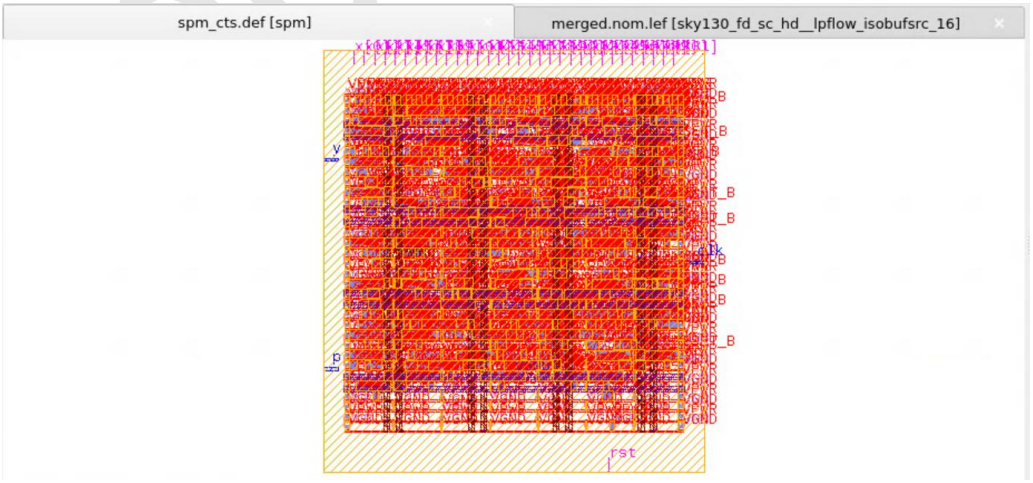


Figure 1: klayout view of Spm_cts.def

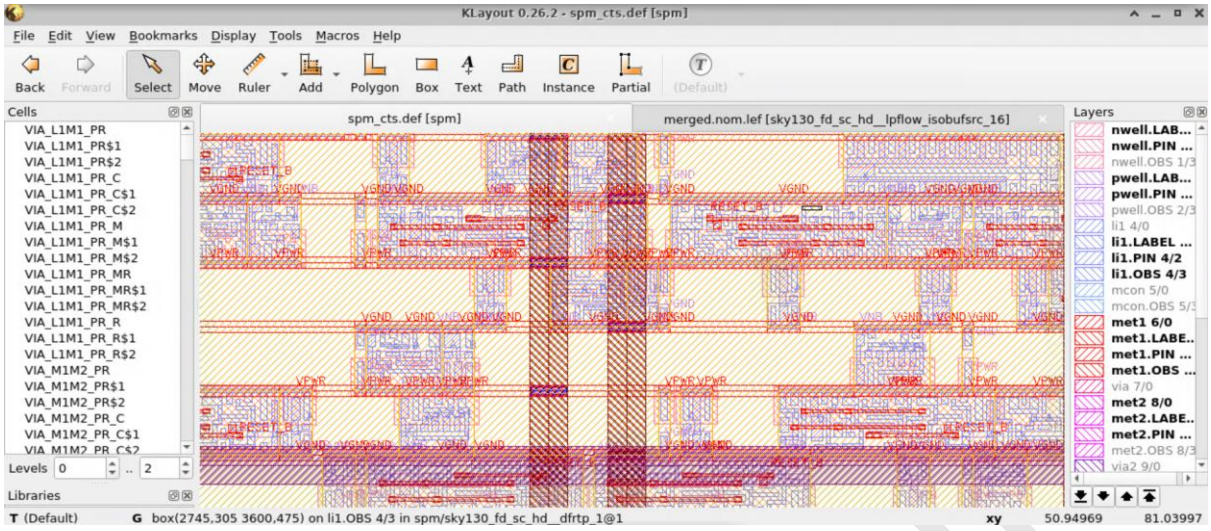


Figure 2: zoom in klayout view of Spm_cts.def

NOTE: To see clock three disable all metal layer except M2 and M3 that is clock routing layer.

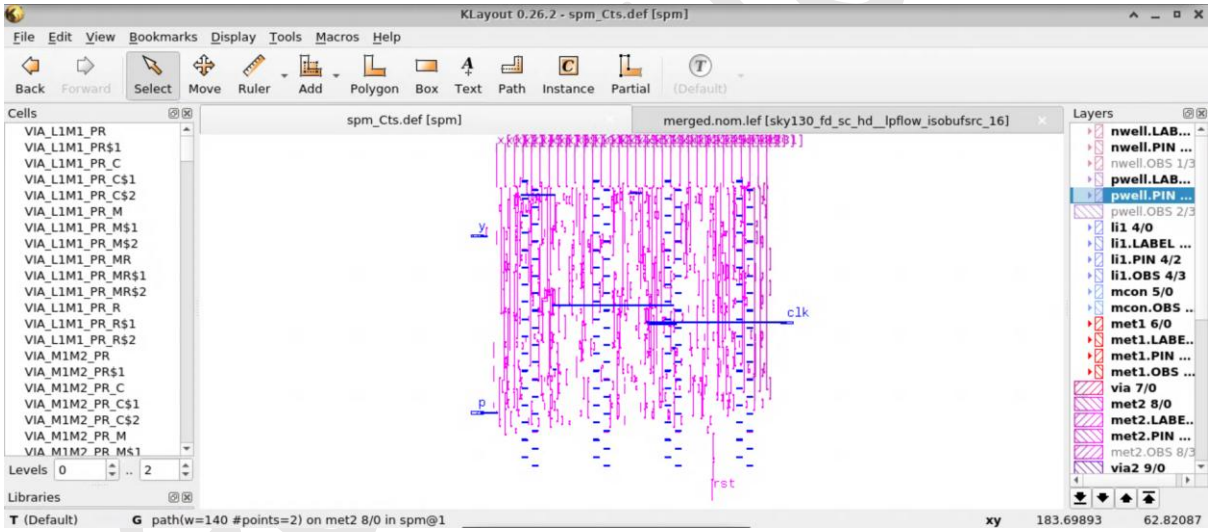


Figure 3 : Zoom in klayout view of clock tree synthesis spm_cts.def

Note: If you are not able to see the clock tree in spm_cts.def, then check it in spm_routing.def after routing; the procedure to view it will remain the same.

OBSERVATION:

- Clock buffer and inverter placement
- Clock tree topology and branching
- Clock skew and insertion delay
- Balanced clock distribution across the core
- No clock violations or disconnected sinks

• CTS DEF visualization in klayout

OUTPUT:

Output of the CTS stage is a spm.def file and reports that show:

• Synthesized clock tree network

• Inserted clock buffers and inverters

• Reduced clock skew and latency

• Clock-connected sequential elements

• CTS-ready design for routing

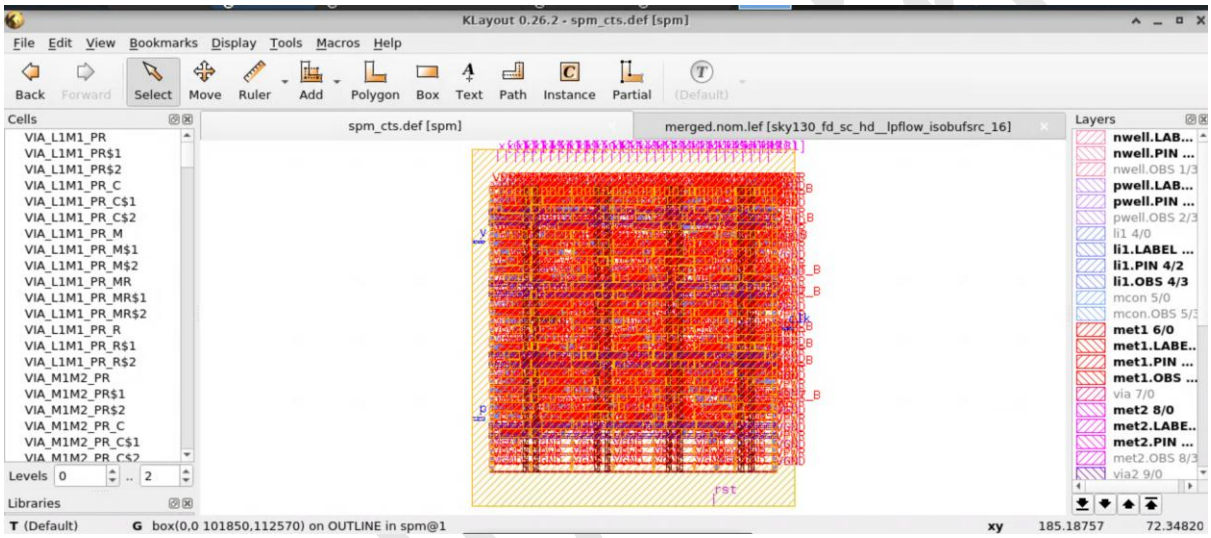


Figure 4: Output of Spm_cts.def

CONCLUSION:

In this CTS lab, OpenLane was successfully used to generate a balanced and optimized clock distribution network for the given RTL design. The clock tree synthesis stage effectively inserted buffers and inverters to minimize skew and latency while meeting timing constraints. CTS results were verified through reports and layout visualization using klayout, confirming proper clock connectivity and distribution. This successful CTS implementation provides a strong foundation for routing and final timing closure, ensuring reliable synchronous operation of the ASIC design